MAAI901 Week 6 experiment notebook (Ahmad Hello, 7573297)

Locating Photos by Country: CLIP ViT-B/16 vs. ImageNet-21k ViT-B/16 vs. ResNet-50

Applied Vision Transformer Report: experiment notebook (MAAI901). Ahmad Hello, 7573297.

Steps: A splits → B image cache → C scene labels → D learning-rate search → E final training → F test evaluation → G results and figures.

In [55]:
```
# ---- Settings ------------------------------------------------------------
SEEDS = [0, 1, 2]                 # final runs per model
MODELS_TO_RUN = ["clip", "vit", "resnet"]
LR_GRID = [1e-5, 3e-5, 1e-4, 3e-4]
LR_EXTEND = {1e-5: 3e-6, 3e-4: 1e-3}     # one extra rate if the best one is at an end of the grid
STAGE1_EPOCHS, STAGE1_LR = 2, 1e-3
STAGE2_EPOCHS, SEARCH_EPOCHS = 8, 3
BATCH = 64
EXPECTED_SPLIT_CHECKSUM = "3ec8dc9f836b"
DRIVE_DIR = "/content/drive/MyDrive/MAAI901_W6"
import logging; logging.getLogger("urllib3").setLevel(logging.ERROR)
```

In [56]:
```
import os, sys, json, time, glob, shutil, subprocess
try:
    gpu = subprocess.run(["nvidia-smi", "--query-gpu=name,memory.total", "--format=csv,noheader"], capture_output=True, text=True).stdout.strip()
except FileNotFoundError:
    gpu = ""
print("GPU:", gpu or "none")
from google.colab import drive
drive.mount("/content/drive")
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "remotezip"], check=True)
for d in ["splits", "cache", "scene", "runs", "models", "preds", "results", "figures"]:
    os.makedirs(f"{DRIVE_DIR}/{d}", exist_ok=True)
os.makedirs("/content/src", exist_ok=True)
sys.path.insert(0, "/content/src")
```

```
GPU: Tesla T4, 15360 MiB
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

In [57]:
```
%%writefile /content/src/data_prep.py
"""Data preparation for the 25-country study (OSV-5M).

Builds the balanced splits from the metadata, maps every selected image to its
archive on Hugging Face, and caches each image once at 256 x 256 (whole
```

```
frame,
bicubic) as JPEG shards.
"""
import io, os, re, json, time, struct, zipfile, hashlib, threading
from concurrent.futures import ThreadPoolExecutor, as_completed

import numpy as np
import pandas as pd
import requests
from PIL import Image

HF = "https://huggingface.co/datasets/osv5m/osv5m/resolve/main"
COUNTRIES = sorted("US CA MX BR AR CL PE NO ES FI SE FR GB ZA ZM BW TZ
RU IN TR TH ID JP AU NZ".split())
LABEL = {c: i for i, c in enumerate(COUNTRIES)}
N_TRAIN, N_VAL, N_TEST = 1500, 200, 1000
CAP_TRAIN, CAP_TEST = 10, 5
N_BACKUP = 40          # spare images per country and split, used only
if a file fails to open
SEED = 42
COLS = ["id", "country", "sequence", "creator_id"]


def load_meta(train_csv, test_csv):
    dt = {c: str for c in COLS}
    tr = pd.read_csv(train_csv, usecols=COLS, dtype=dt)
    te = pd.read_csv(test_csv, usecols=COLS, dtype=dt)
    tr = tr[tr.country.isin(COUNTRIES)].reset_index(drop=True)
    te = te[te.country.isin(COUNTRIES)].reset_index(drop=True)
    return tr, te


def _draw(groups, order, cap, n_needed, n_backup, rng, taken):
    """Walk sequences in `order`, taking random images from each until
n_needed picks
    (then n_backup spares) are collected. `taken` counts images already
drawn from each
    sequence across all countries, so a sequence that crosses a border
still respects the cap."""
    picks, spares = [], []
    for s in order:
        if len(picks) >= n_needed and len(spares) >= n_backup:
            break
        room = cap - taken.get(s, 0)
        if room <= 0:
            continue
        ids = groups[s]
        chosen = list(rng.choice(ids, size=min(room, len(ids)),
replace=False))
        for i in chosen:
            if len(picks) < n_needed:
                picks.append(i)
            elif len(spares) < n_backup:
                spares.append(i)
            else:
                break
            taken[s] = taken.get(s, 0) + 1
    return picks, spares
```

```
def build_splits(tr, te, seed=SEED):
    rngs = {c: np.random.RandomState(seed + LABEL[c]) for c in
COUNTRIES}
    tg = {c: {s: sorted(g.id.tolist()) for s, g in tr[tr.country ==
c].groupby("sequence", sort=True)} for c in COUNTRIES}
    eg = {c: {s: sorted(g.id.tolist()) for s, g in te[te.country ==
c].groupby("sequence", sort=True)} for c in COUNTRIES}
    out, val_seqs = {}, set()
    # 1) validation: whole sequences, at most 10 images each; Botswana
uses short sequences only.
    #    Every sequence touched here is removed from training (in all
countries).
    for c in COUNTRIES:
        rng, groups = rngs[c], tg[c]
        cand = [s for s in sorted(groups) if c != "BW" or len(groups[s])
<= CAP_TRAIN]
        val, val_sp = [], []
        for s in rng.permutation(cand):
            if len(val) >= N_VAL and len(val_sp) >= 10:
                break
            ids = groups[s]
            val_seqs.add(s)
            for i in rng.choice(ids, size=min(CAP_TRAIN, len(ids)),
replace=False):
                if len(val) < N_VAL:
                    val.append(i)
                elif len(val_sp) < 10:
                    val_sp.append(i)
        out[c] = {"val": val, "val_spare": val_sp}
    # 2) training: remaining sequences, at most 10 images per sequence
(cap lifted for Botswana)
    taken = {}
    for c in COUNTRIES:
        rng = rngs[c]
        pool = {s: ids for s, ids in tg[c].items() if s not in val_seqs}
        if c == "BW":
            perm = list(rng.permutation(sorted(i for ids in
pool.values() for i in ids)))
            trn, trn_sp = perm[:N_TRAIN], perm[N_TRAIN:N_TRAIN +
N_BACKUP]
        else:
            trn, trn_sp = _draw(pool,
list(rng.permutation(sorted(pool))), CAP_TRAIN, N_TRAIN, N_BACKUP, rng,
taken)
        out[c].update(train=trn, train_spare=trn_sp)
    # 3) test: official test set, at most 5 images per sequence
    taken_t = {}
    for c in COUNTRIES:
        rng, groups = rngs[c], eg[c]
        tst, tst_sp = _draw(groups,
list(rng.permutation(sorted(groups))), CAP_TEST, N_TEST, N_BACKUP, rng,
taken_t)
        out[c].update(test=tst, test_spare=tst_sp)
    rows = []
    for c in COUNTRIES:
        assert len(out[c]["val"]) == N_VAL and len(out[c]["train"]) ==
N_TRAIN and len(out[c]["test"]) == N_TEST, c
        for split in ["train", "train_spare", "val", "val_spare",
"test", "test_spare"]:
            for rank, i in enumerate(out[c][split]):
```

```
                rows.append((i, split, c, LABEL[c], rank))
    df = pd.DataFrame(rows, columns=["id", "split", "country", "label",
"rank"])
    meta = pd.concat([tr.assign(src="train"), te.assign(src="test")])
    df = df.merge(meta[["id", "sequence", "creator_id", "src"]],
on="id", how="left")
    assert df.sequence.notna().all() and not df.id.duplicated().any()
    return df


def split_checksum(df):
    core = df[df.split.isin(["train", "val",
"test"])].sort_values(["split", "id"])
    return hashlib.md5(("|".join(core.split + ":" +
core.id)).encode()).hexdigest()[:12]


# ---------------------------------------------------------------
archive index
def _final_url(sess, url):
    r = sess.head(url, allow_redirects=True, timeout=60)
    return r.url if r.status_code == 200 else url


def list_archive(url):
    from remotezip import RemoteZip
    with RemoteZip(url) as z:
        out = []
        for i in z.infolist():
            if i.filename.endswith(".jpg"):
                out.append((os.path.basename(i.filename)[:-4],
i.header_offset, i.compress_size, i.compress_type))
    return out


def build_index(needed_ids, which, n_archives, workers=8, log=print):
    """Map each needed id to (archive url, offset, size,
compression)."""
    needed = set(needed_ids)
    urls = [f"{HF}/images/{which}/{k:02d}.zip" for k in
range(n_archives)]
    found = {}
    def job(k):
        return k, list_archive(urls[k])
    with ThreadPoolExecutor(workers) as ex:
        for k, entries in ex.map(job, range(n_archives)):
            for i, off, size, comp in entries:
                if i in needed:
                    found[i] = (k, off, size, comp)
    log(f"{which}: indexed {len(found)}/{len(needed)} images from
{n_archives} archives")
    return found, urls


# ---------------------------------------------------------------- fetch
+ cache
_local = threading.local()

def _session():
    if not hasattr(_local, "s"):
```

```
        _local.s = requests.Session()
    return _local.s


def fetch_bytes(url, off, size, comp, tries=4):
    for t in range(tries):
        try:
            r = _session().get(url, headers={"Range": f"bytes={off}-{off
+ 30 + 512 + size}"}, timeout=60)
            b = r.content
            sig, _, _, cm, _, _, _, _, _, nl, el = struct.unpack("
<IHHHHHIIIHH", b[:30])
            if sig != 0x04034B50:
                raise ValueError("bad header")
            data = b[30 + nl + el: 30 + nl + el + size]
            if cm == 8:
                import zlib
                data = zlib.decompress(data, -15)
            return data
        except Exception:
            time.sleep(1 + 2 * t)
    return None


def to_cache_jpeg(data, side=256, quality=90):
    im = Image.open(io.BytesIO(data))
    im.draft("RGB", (side, side))                       # fast JPEG
decode at reduced scale (still >= side)
    im = im.convert("RGB").resize((side, side), Image.BICUBIC)   # whole
frame, no crop
    buf = io.BytesIO()
    im.save(buf, "JPEG", quality=quality)
    return buf.getvalue()


def cache_split(ids, index, urls, out_dir, prefix, shard_size=2500,
workers=48, log=print):
    """Download, resize and store images in shard zips; returns ids that
failed.
    Finished shards are skipped."""
    os.makedirs(out_dir, exist_ok=True)
    final = {}
    sess = requests.Session()
    failed = []
    shards = [ids[i:i + shard_size] for i in range(0, len(ids),
shard_size)]
    for si, shard in enumerate(shards):
        path = os.path.join(out_dir, f"{prefix}_{si:03d}.zip")
        if os.path.exists(path):
            continue
        t0 = time.time()
        tmp = f"/tmp/{prefix}_{si:03d}.zip"
        with zipfile.ZipFile(tmp, "w", zipfile.ZIP_STORED) as zf,
ThreadPoolExecutor(workers) as ex:
            def job(i):
                k, off, size, comp = index[i]
                if k not in final:
                    final[k] = _final_url(sess, urls[k])
                data = fetch_bytes(final[k], off, size, comp)
                if data is None:
```

```
                    return i, None
                try:
                    return i, to_cache_jpeg(data)
                except Exception:
                    return i, None
            for fut in as_completed([ex.submit(job, i) for i in shard if
i in index]):
                i, jpg = fut.result()
                if jpg is None:
                    failed.append(i)
                else:
                    zf.writestr(f"{i}.jpg", jpg)
        missing = [i for i in shard if i not in index]
        failed += missing
        import shutil
        shutil.move(tmp, path)
        log(f"{prefix} shard {si + 1}/{len(shards)}: {len(shard)} images
in {time.time() - t0:.0f}s")
    return failed


def cached_ids(out_dir, prefix):
    have = set()
    for f in sorted(os.listdir(out_dir)):
        if f.startswith(prefix + "_") and f.endswith(".zip"):
            with zipfile.ZipFile(os.path.join(out_dir, f)) as z:
                have.update(n[:-4] for n in z.namelist())
    return have
```

```
Overwriting /content/src/data_prep.py
```

In [58]:

```
%%writefile /content/src/train_eval.py
"""Models, data loading, scene labels, training and evaluation for the
study."""
import io, os, json, math, time, random, zipfile
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from PIL import Image, ImageEnhance

MODELS = {
    "clip":   dict(ckpt="openai/clip-vit-base-patch16",
                   mean=(0.48145466, 0.4578275, 0.40821073), std=
(0.26862954, 0.26130258, 0.27577711), feat=768),
    "vit":    dict(ckpt="google/vit-base-patch16-224-in21k", mean=(0.5,
0.5, 0.5), std=(0.5, 0.5, 0.5), feat=768),
    "resnet": dict(ckpt="microsoft/resnet-50", mean=(0.485, 0.456,
0.406), std=(0.229, 0.224, 0.225), feat=2048),
}
NAMES = {"clip": "CLIP ViT-B/16", "vit": "ViT-B/16 (IN-21k)", "resnet":
"ResNet-50"}
N_CLASSES = 25


# ---------------------------------------------------------------------
data
class ShardStore:
    """Reads cached JPEGs from shard zips; each worker opens its own
handles."""
```

```
    def __init__(self, shard_paths):
        self.paths = list(shard_paths)
        self.where = {}
        for p in self.paths:
            with zipfile.ZipFile(p) as z:
                for n in z.namelist():
                    self.where[n[:-4]] = p
        self._open, self._pid = {}, os.getpid()

    def get(self, i):
        if os.getpid() != self._pid:          # new DataLoader worker:
never share file handles across processes
            self._open, self._pid = {}, os.getpid()
        p = self.where[i]
        z = self._open.get(p)
        if z is None:
            z = self._open[p] = zipfile.ZipFile(p)
        return Image.open(io.BytesIO(z.read(f"{i}.jpg"))).convert("RGB")

    def __getstate__(self):
        d = dict(self.__dict__); d["_open"] = {}; return d


def random_resized_crop(im, size=224, scale=(0.8, 1.0), ratio=(3 / 4, 4
/ 3), rng=random):
    W, H = im.size
    area = W * H
    for _ in range(10):
        a = area * rng.uniform(*scale)
        r = math.exp(rng.uniform(math.log(ratio[0]),
math.log(ratio[1])))
        w, h = int(round(math.sqrt(a * r))), int(round(math.sqrt(a /
r)))
        if 0 < w <= W and 0 < h <= H:
            x, y = rng.randint(0, W - w), rng.randint(0, H - h)
            return im.crop((x, y, x + w, y + h)).resize((size, size),
Image.BICUBIC)
    return im.resize((size, size), Image.BICUBIC)


def to_tensor(im, mean, std):
    a = np.asarray(im, dtype=np.float32) / 255.0
    a = (a - np.array(mean, dtype=np.float32)) / np.array(std,
dtype=np.float32)
    return torch.from_numpy(a.transpose(2, 0, 1).copy())


class ImageSet(torch.utils.data.Dataset):
    def __init__(self, store, ids, labels, mean, std, train=False,
center_crop=False):
        self.store, self.ids, self.labels = store, list(ids),
list(labels)
        self.mean, self.std, self.train, self.center_crop = mean, std,
train, center_crop

    def __len__(self):
        return len(self.ids)

    def __getitem__(self, k):
        im = self.store.get(self.ids[k])
```

```
        if self.train:
            im = random_resized_crop(im)
            im = ImageEnhance.Brightness(im).enhance(random.uniform(0.9,
1.1))
            im = ImageEnhance.Contrast(im).enhance(random.uniform(0.9,
1.1))
        elif self.center_crop:           # used only for the Places365
scene classifier
            im = im.crop((16, 16, 240, 240))
        else:
            im = im.resize((224, 224), Image.BICUBIC)
        return to_tensor(im, self.mean, self.std), self.labels[k]


def _worker_init(wid):
    s = torch.initial_seed() % 2**31
    random.seed(s + wid); np.random.seed((s + wid) % 2**31)


def loader(ds, batch, shuffle, seed=0, workers=2, drop_last=False):
    g = torch.Generator(); g.manual_seed(seed)
    return torch.utils.data.DataLoader(ds, batch_size=batch,
shuffle=shuffle, generator=g, num_workers=workers,

pin_memory=torch.cuda.is_available(), drop_last=drop_last,
                                       worker_init_fn=_worker_init,
persistent_workers=False)


# ---------------------------------------------------------------------
models
class Net(nn.Module):
    def __init__(self, kind):
        super().__init__()
        from transformers import CLIPVisionModel, ViTModel, ResNetModel
        self.kind = kind
        ck = MODELS[kind]["ckpt"]
        if kind == "clip":
            self.backbone = CLIPVisionModel.from_pretrained(ck)
        elif kind == "vit":
            self.backbone = ViTModel.from_pretrained(ck,
add_pooling_layer=False)
        else:
            self.backbone = ResNetModel.from_pretrained(ck)
        self.head = nn.Linear(MODELS[kind]["feat"], N_CLASSES)

    def features(self, x):
        if self.kind == "clip":
            return self.backbone(pixel_values=x).pooler_output        #
CLS after final layer norm, before projection
        if self.kind == "vit":
            return self.backbone(pixel_values=x).last_hidden_state[:, 0]
# class token
        return self.backbone(pixel_values=x).pooler_output.flatten(1)
# 2,048-d pooled features

    def forward(self, x, frozen=False):
        if frozen:
            with torch.no_grad():
                f = self.features(x)
```

```
            return self.head(f.float())
        return self.head(self.features(x).float())


def param_groups(model, wd):
    decay, no_decay = [], []
    for n, p in model.named_parameters():
        if not p.requires_grad:
            continue
        (no_decay if p.ndim < 2 or "norm" in n.lower() or
n.endswith(".bias") else decay).append(p)
    return [{"params": decay, "weight_decay": wd}, {"params": no_decay,
"weight_decay": 0.0}]


def warmup_cosine(opt, total, warm_frac=0.05):
    warm = max(1, int(total * warm_frac))
    def f(step):
        if step < warm:
            return (step + 1) / warm
        p = (step - warm) / max(1, total - warm)
        return 0.5 * (1 + math.cos(math.pi * min(1.0, p)))
    return torch.optim.lr_scheduler.LambdaLR(opt, f)


@torch.no_grad()
def predict(model, dl, device, frozen=False):
    model.eval()
    out, ys = [], []
    for x, y in dl:
        x = x.to(device, non_blocking=True)
        with torch.autocast("cuda", dtype=torch.float16,
enabled=device.type == "cuda"):
            out.append(model(x).float().cpu())
        ys.append(y)
    return torch.cat(out).numpy(), torch.cat(ys).numpy()


def set_seed(s):
    random.seed(s); np.random.seed(s); torch.manual_seed(s);
torch.cuda.manual_seed_all(s)


def run_stage(model, stage, train_dl_fn, val_dl, epochs, lr, device,
ckpt_path=None, log=print,
              wd=0.05, clip=1.0, start_state=None):
    """One training stage. stage 1 = head only on a frozen backbone,
stage 2 = all weights.
    Saves a checkpoint after every epoch (replacing the previous one).
    Returns per-epoch validation accuracy and the state dict of the best
epoch."""
    frozen = stage == 1
    model.backbone.requires_grad_(not frozen)
    opt = torch.optim.AdamW(param_groups(model, wd), lr=lr, betas=(0.9,
0.999))
    steps_per_epoch = len(train_dl_fn(0))
    sched = warmup_cosine(opt, epochs * steps_per_epoch)
    scaler = torch.amp.GradScaler("cuda", enabled=device.type == "cuda")
    hist, best, best_state, start_ep = [], -1.0, None, 0
    if ckpt_path and os.path.exists(ckpt_path):
```

```
        st = torch.load(ckpt_path, map_location="cpu",
weights_only=False)
        model.load_state_dict(st["model"]);
opt.load_state_dict(st["opt"]); sched.load_state_dict(st["sched"])
        scaler.load_state_dict(st["scaler"]); hist, best, start_ep =
st["hist"], st["best"], st["epoch"] + 1
        best_state = st.get("best_state")
        log(f"  resumed at epoch {start_ep + 1}/{epochs}")
    for ep in range(start_ep, epochs):
        t0 = time.time()
        if frozen:
            model.eval(); model.head.train()
        else:
            model.train()
        tot, n = 0.0, 0
        for x, y in train_dl_fn(ep):
            x, y = x.to(device, non_blocking=True), y.to(device,
non_blocking=True)
            with torch.autocast("cuda", dtype=torch.float16,
enabled=device.type == "cuda"):
                loss = F.cross_entropy(model(x, frozen=frozen), y)
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.unscale_(opt)
            nn.utils.clip_grad_norm_([p for p in model.parameters() if
p.requires_grad], clip)
            scaler.step(opt); scaler.update(); sched.step()
            tot += loss.item() * len(y); n += len(y)
        logits, ys = predict(model, val_dl, device)
        acc = float((logits.argmax(1) == ys).mean())
        hist.append({"epoch": ep + 1, "train_loss": tot / n, "val_acc":
acc, "minutes": (time.time() - t0) / 60})
        log(f"  stage {stage} epoch {ep + 1}/{epochs}: loss {tot /
n:.3f}  val acc {acc:.4f}  ({(time.time() - t0) / 60:.1f} min)")
        if acc > best:
            best = acc
            best_state = {k: v.detach().half().cpu() if
v.is_floating_point() else v.cpu() for k, v in
model.state_dict().items()} \
                if stage == 2 else {k: v.detach().cpu() for k, v in
model.head.state_dict().items()}
        if ckpt_path:
            torch.save({"model": model.state_dict(), "opt":
opt.state_dict(), "sched": sched.state_dict(),
                        "scaler": scaler.state_dict(), "hist": hist,
"best": best, "epoch": ep,
                        "best_state": best_state}, ckpt_path + ".tmp")
            os.replace(ckpt_path + ".tmp", ckpt_path)
    return hist, best_state


# ---------------------------------------------------------------------
Places365 scene classifier
class Bottleneck(nn.Module):
    expansion = 4
    def __init__(self, inp, planes, stride=1, down=None):
        super().__init__()
        self.conv1 = nn.Conv2d(inp, planes, 1, bias=False); self.bn1 =
nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, 3, stride, 1,
```

```
bias=False); self.bn2 = nn.BatchNorm2d(planes)
        self.conv3 = nn.Conv2d(planes, planes * 4, 1, bias=False);
self.bn3 = nn.BatchNorm2d(planes * 4)
        self.relu = nn.ReLU(inplace=True); self.downsample = down
    def forward(self, x):
        idt = x if self.downsample is None else self.downsample(x)
        x = self.relu(self.bn1(self.conv1(x))); x =
self.relu(self.bn2(self.conv2(x))); x = self.bn3(self.conv3(x))
        return self.relu(x + idt)


class ResNet50(nn.Module):
    """Same layout and parameter names as torchvision's resnet50, so the
Places365 weights load directly."""
    def __init__(self, n=365):
        super().__init__()
        self.inp = 64
        self.conv1 = nn.Conv2d(3, 64, 7, 2, 3, bias=False); self.bn1 =
nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True); self.maxpool =
nn.MaxPool2d(3, 2, 1)
        self.layer1 = self._make(64, 3); self.layer2 = self._make(128,
4, 2)
        self.layer3 = self._make(256, 6, 2); self.layer4 =
self._make(512, 3, 2)
        self.avgpool = nn.AdaptiveAvgPool2d(1); self.fc =
nn.Linear(2048, n)
    def _make(self, planes, blocks, stride=1):
        down = None
        if stride != 1 or self.inp != planes * 4:
            down = nn.Sequential(nn.Conv2d(self.inp, planes * 4, 1,
stride, bias=False), nn.BatchNorm2d(planes * 4))
        layers = [Bottleneck(self.inp, planes, stride, down)]
        self.inp = planes * 4
        layers += [Bottleneck(self.inp, planes) for _ in range(1,
blocks)]
        return nn.Sequential(*layers)
    def forward(self, x):
        x = self.maxpool(self.relu(self.bn1(self.conv1(x))))
        x = self.layer4(self.layer3(self.layer2(self.layer1(x))))
        return self.fc(torch.flatten(self.avgpool(x), 1))


def load_places365(path):
    m = ResNet50()
    st = torch.load(path, map_location="cpu", weights_only=False)
    st = st.get("state_dict", st)
    m.load_state_dict({k.replace("module.", ""): v for k, v in
st.items()})
    return m.eval()


def scene_groups(probs, W):
    """probs: (N, 365) softmax; W: (365, 3) membership (indoor, natural,
man-made).
    A category in several groups is split equally between them, so the
three totals sum to one."""
    Wn = W / W.sum(1, keepdims=True)
    g = probs @ Wn
```

```
    lab = np.array(["indoor", "natural", "urban"])[g.argmax(1)]
    return g, lab
```

```
Overwriting /content/src/train_eval.py
```

In [59]:

```
%%writefile /content/src/analysis.py
"""Evaluation and statistics: metrics, McNemar (H1), sequence bootstrap
(H2), figures."""
import json, math
import numpy as np
import pandas as pd
from scipy.stats import chi2, binomtest
from sklearn.metrics import f1_score, confusion_matrix

LOOKALIKE = [["US", "CA"], ["AR", "CL"], ["NO", "SE", "FI"], ["ZA",
"BW", "ZM"], ["TH", "ID"], ["AU", "NZ"]]


def mcnemar(correct_a, correct_b):
    """Continuity-corrected McNemar test on paired predictions
(Dietterich, 1998)."""
    a, b = np.asarray(correct_a, bool), np.asarray(correct_b, bool)
    n01, n10 = int((a & ~b).sum()), int((~a & b).sum())
    stat = (abs(n01 - n10) - 1) ** 2 / max(1, n01 + n10)
    return {"a_only": n01, "b_only": n10, "chi2": stat, "p":
float(chi2.sf(stat, 1)),
            "p_exact": float(binomtest(n01, n01 + n10, 0.5).pvalue) if
n01 + n10 > 0 else 1.0}


def basic_metrics(pred, y, countries):
    acc = float((pred == y).mean())
    f1 = float(f1_score(y, pred, average="macro"))
    rec = {c: float((pred[y == k] == k).mean()) for k, c in
enumerate(countries)}
    return {"top1": acc, "macro_f1": f1, "recall": rec}


def scene_table(test, preds, models, seeds, mask=None):
    """Urban and natural accuracy per model, seed and country."""
    rows = []
    t = test if mask is None else test[mask]
    for m in models:
        for s in seeds:
            ok = preds[(m, s)][t.index.values] == t.label.values
            for c, g in t.assign(ok=ok).groupby("country"):
                for sc in ["urban", "natural"]:
                    gg = g[g.scene == sc]
                    rows.append((m, s, c, sc, len(gg), gg.ok.mean() if
len(gg) else np.nan))
    return pd.DataFrame(rows, columns=["model", "seed", "country",
"scene", "n", "acc"])


def eligible_countries(test, mask=None, k=50):
    t = test if mask is None else test[mask]
    n = t.groupby(["country", "scene"]).size().unstack(fill_value=0)
    for col in ["urban", "natural"]:
        if col not in n:
            n[col] = 0
```

```
    return sorted(n[(n.urban >= k) & (n.natural >= k)].index.tolist())


def _drop_from_weights(ok, w, country_idx, urban, natural, keep, pooled,
n_c):
    if pooled:
        u = (ok * w * urban).sum() / max(1e-9, (w * urban).sum())
        n = (ok * w * natural).sum() / max(1e-9, (w * natural).sum())
        return 100 * (u - n)
    cu = np.bincount(country_idx, ok * w * urban, n_c) / np.maximum(1e-
9, np.bincount(country_idx, w * urban, n_c))
    cn = np.bincount(country_idx, ok * w * natural, n_c) /
np.maximum(1e-9, np.bincount(country_idx, w * natural, n_c))
    return 100 * float(np.mean((cu - cn)[keep]))


def drop_estimate(test, preds, model, seeds, countries, pooled,
weights=None):
    """Urban minus natural accuracy (percentage points): per eligible
country, averaged over
    countries (or on the pooled set), then averaged over seeds. Returns
(mean, per-seed list)."""
    t = test.reset_index(drop=True)
    names = sorted(t.country.unique())
    ci = t.country.map({c: k for k, c in enumerate(names)}).values
    keep = np.array([c in countries for c in names])
    urban = (t.scene.values == "urban").astype(float); natural =
(t.scene.values == "natural").astype(float)
    w = np.ones(len(t)) if weights is None else weights
    per = [_drop_from_weights((preds[(model, s)] ==
t.label.values).astype(float), w, ci, urban, natural, keep,
                              pooled, len(names)) for s in seeds]
    return float(np.mean(per)), per


def h2_bootstrap(test, preds, seeds, countries, pooled, B=2000, seed=0,
a="clip", b="vit"):
    """95% interval for drop(a) - drop(b), resampling whole test
sequences within each country.
    The same resamples are applied to every seed and both models."""
    rng = np.random.RandomState(seed)
    t = test.reset_index(drop=True)
    names = sorted(t.country.unique())
    ci = t.country.map({c: k for k, c in enumerate(names)}).values
    keep = np.array([c in countries for c in names])
    urban = (t.scene.values == "urban").astype(float); natural =
(t.scene.values == "natural").astype(float)
    oks = {(m, s): (preds[(m, s)] == t.label.values).astype(float) for m
in (a, b) for s in seeds}
    # per country: local sequence index of every row
    blocks = []
    for c in names:
        rows = np.where(t.country.values == c)[0]
        codes, local = np.unique(t.sequence.values[rows],
return_inverse=True)
        blocks.append((rows, local, len(codes)))
    diffs = np.empty(B)
    for i in range(B):
        w = np.zeros(len(t))
        for rows, local, n in blocks:
```

```
            cnt = np.bincount(rng.randint(0, n, n), minlength=n)
            w[rows] = cnt[local]
        da = np.mean([_drop_from_weights(oks[(a, s)], w, ci, urban,
natural, keep, pooled, len(names)) for s in seeds])
        db = np.mean([_drop_from_weights(oks[(b, s)], w, ci, urban,
natural, keep, pooled, len(names)) for s in seeds])
        diffs[i] = da - db
    return {"lo": float(np.percentile(diffs, 2.5)), "hi":
float(np.percentile(diffs, 97.5)), "samples": diffs}


def top_confusions(pred, y, countries, k=10):
    cm = confusion_matrix(y, pred, labels=range(len(countries)))
    off = [(cm[i, j], countries[i], countries[j]) for i in
range(len(countries)) for j in range(len(countries)) if i != j]
    return sorted(off, reverse=True)[:k], cm


def lookalike_share(pred, y, countries):
    """Share of errors that stay inside a look-alike group."""
    gid = {}
    for g, members in enumerate(LOOKALIKE):
        for c in members:
            gid[c] = g
    err = pred != y
    inside = [gid.get(countries[a], -1) == gid.get(countries[b], -2) for
a, b in zip(y[err], pred[err])]
    return float(np.mean(inside)) if len(inside) else float("nan")
```

```
Overwriting /content/src/analysis.py
```

In [60]:

```
import importlib, numpy as np, pandas as pd, torch
import data_prep as dp, train_eval as te, analysis as an
for m in (dp, te, an): importlib.reload(m)
import transformers
transformers.logging.set_verbosity_error()
transformers.logging.disable_progress_bar()
print("torch", torch.__version__, "| transformers",
transformers.__version__)
log_path = f"{DRIVE_DIR}/results/log.txt"
def log(*a):
    msg = " ".join(str(x) for x in a)
    print(msg, flush=True)
    with open(log_path, "a") as f:
        f.write(time.strftime("%Y-%m-%d %H:%M ") + msg + "\n")
```

```
torch 2.11.0+cu130 | transformers 5.20.2
```

A. Balanced splits (1,500 / 200 / 1,000 images per country)

In [61]:

```
split_file = f"{DRIVE_DIR}/splits/splits.csv.gz"
if not os.path.exists(split_file):
    from huggingface_hub import hf_hub_download
    t0 = time.time()
    tr_csv = hf_hub_download("osv5m/osv5m", "train.csv",
repo_type="dataset", local_dir="/content/meta")
    te_csv = hf_hub_download("osv5m/osv5m", "test.csv",
repo_type="dataset", local_dir="/content/meta")
```

```
    tr, te_ = dp.load_meta(tr_csv, te_csv)
    log(f"metadata: {len(tr):,} training and {len(te_):,} test images
for the 25 countries ({time.time() - t0:.0f}s)")
    splits = dp.build_splits(tr, te_)
    splits.to_csv(split_file, index=False)
    del tr, te_
splits = pd.read_csv(split_file, dtype=str)
splits["label"] = splits.label.astype(int)
chk = dp.split_checksum(splits)
log("split checksum", chk, "(matches the planned splits)" if chk ==
EXPECTED_SPLIT_CHECKSUM else "(DIFFERENT from expected)")
print(splits[splits.split.isin(["train", "val",
"test"])].groupby("split").size())
```

```
split checksum 3ec8dc9f836b (matches the planned splits)
split
test     25000
train    37500
val       5000
dtype: int64
```

B. Image cache: each selected image is read from the online archives once and stored at 256 × 256 (whole frame)

In [62]:

```
final_file = f"{DRIVE_DIR}/splits/splits_final.csv.gz"
CACHE = f"{DRIVE_DIR}/cache"
if not os.path.exists(final_file):
    need = splits
    idx_tr, urls_tr = dp.build_index(need[need.src == "train"].id,
"train", 98, log=log)
    idx_te, urls_te = dp.build_index(need[need.src == "test"].id,
"test", 5, log=log)
    core = splits[splits.split.isin(["train", "val", "test"])]
    failed = []
    for split, src in [("val", "train"), ("test", "test"), ("train",
"train")]:
        ids = core[core.split == split].id.tolist()
        idx, urls = (idx_tr, urls_tr) if src == "train" else (idx_te,
urls_te)
        t0 = time.time()
        failed += dp.cache_split(ids, idx, urls, CACHE, split, log=log)
        log(f"{split}: cached in {(time.time() - t0) / 60:.1f} min")
    # replace any image that could not be read by the next spare from
the same country and split
    final = core.copy()
    if failed:
        log(f"{len(failed)} images failed to open; replacing them with
spares")
        bad = final[final.id.isin(failed)]
        repl = []
        for (split, c), g in bad.groupby(["split", "country"]):
            spares = splits[(splits.split == split + "_spare") &
(splits.country == c)].sort_values("rank", key=lambda s: s.astype(int))
            repl.append(spares.head(len(g) + 5))
        repl = pd.concat(repl)
        for split, src in [("val", "train"), ("test", "test"), ("train",
"train")]:
```

```
            r = repl[repl.split == split + "_spare"]
            if len(r):
                idx, urls = (idx_tr, urls_tr) if src == "train" else
(idx_te, urls_te)
                still = dp.cache_split(r.id.tolist(), idx, urls, CACHE,
split + "_extra", log=log)
                repl = repl[~repl.id.isin(still)]
        final = final[~final.id.isin(failed)]
        for (split, c), g in bad.groupby(["split", "country"]):
            add = repl[(repl.split == split + "_spare") & (repl.country
== c)].head(len(g)).assign(split=split)
            final = pd.concat([final, add])
    final.to_csv(final_file, index=False)
final = pd.read_csv(final_file, dtype=str); final["label"] =
final.label.astype(int)
print(final.groupby(["split"]).size(), "\nper country:",
final.groupby(["split",
"country"]).size().unstack().min(axis=1).to_dict())
```

```
split
test     25000
train    37500
val       5000
dtype: int64
per country: {'test': 1000, 'train': 1500, 'val': 200}
```

In [63]:
```
# copy the cache from Drive to the Colab disk (faster reads during
training)
LOCAL = "/content/cache"
os.makedirs(LOCAL, exist_ok=True)
t0 = time.time()
for f in sorted(glob.glob(f"{CACHE}/*.zip")):
    dst = f"{LOCAL}/{os.path.basename(f)}"
    if not os.path.exists(dst) or os.path.getsize(dst) !=
os.path.getsize(f):
        shutil.copy(f, dst)
store = te.ShardStore(sorted(glob.glob(f"{LOCAL}/*.zip")))
missing = set(final.id) - set(store.where)
print(f"cache: {len(store.where):,} images on local disk ({time.time() -
t0:.0f}s); missing: {len(missing)}")
assert not missing
part = {s: final[final.split == s].reset_index(drop=True) for s in
["train", "val", "test"]}
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
cache: 67,500 images on local disk (1s); missing: 0
```

C. Scene labels (Places365 ResNet-50) for the urban / natural split

In [64]:
```
scene_file = f"{DRIVE_DIR}/scene/scene_labels.csv"
GROUP_CODES =
"144411411434114444144444144444241144244431144441214441444313443444111445421411214241141144414424424444412341112213442211444141414444414414143211134441223314411141423412441143134414214114224123112441411144221141444114141411412411411123244144441241414241143411211441431434121141121244414413221331141441414413211214111414414442241144145314442332242343132141122242313 41"   # per Places365 category: 4 = indoor, 2 = outdoor natural, 1 = outdoor man-made (summed)
```

```
W = np.array([[(int(d) >> 2) & 1, (int(d) >> 1) & 1, int(d) & 1] for d
in GROUP_CODES], dtype=float)
if not os.path.exists(scene_file):
    import urllib.request
    wpath = "/content/resnet50_places365.pth.tar"
    if not os.path.exists(wpath):

urllib.request.urlretrieve("http://places2.csail.mit.edu/models_places36
5/resnet50_places365.pth.tar", wpath)
    pm = te.load_places365(wpath).to(device)
    rows = []
    for s in ["val", "test"]:
        d = part[s]
        ds = te.ImageSet(store, d.id, d.label, (0.485, 0.456, 0.406),
(0.229, 0.224, 0.225), center_crop=True)
        probs = []
        with torch.no_grad():
            for x, _ in te.loader(ds, 256, False):
                probs.append(torch.softmax(pm(x.to(device)).float(),
1).cpu().numpy())
        g, lab = te.scene_groups(np.concatenate(probs), W)
        rows.append(pd.DataFrame({"id": d.id, "split": s, "p_indoor":
g[:, 0], "p_natural": g[:, 1], "p_urban": g[:, 2], "scene": lab}))
    pd.concat(rows).to_csv(scene_file, index=False)
    del pm
scene = pd.read_csv(scene_file, dtype={"id": str})
scene["conf"] = scene[["p_indoor", "p_natural", "p_urban"]].max(axis=1)
print("Check 1 - group sizes (share of images):")
print(scene.groupby("split").scene.value_counts(normalize=True).unstack(
).round(3))
test = part["test"].merge(scene[scene.split == "test"][["id", "scene",
"conf"]], on="id", how="left")
ELIGIBLE = an.eligible_countries(test)
log("H2 eligible countries (>=50 urban and >=50 natural test images):",
len(ELIGIBLE), ELIGIBLE)
json.dump({"eligible": ELIGIBLE}, open(f"
{DRIVE_DIR}/scene/eligible.json", "w"))
```

```
Check 1 - group sizes (share of images):
scene  indoor  natural  urban
split
test    0.004    0.839  0.157
val     0.006    0.548  0.446
H2 eligible countries (>=50 urban and >=50 natural test images): 21 ['AR',
'AU', 'BR', 'CA', 'CL', 'ES', 'FR', 'GB', 'ID', 'IN', 'JP', 'MX', 'NO',
'NZ', 'PE', 'RU', 'SE', 'TH', 'TR', 'TZ', 'US']
```

In [65]:

```
# Check 2 - contact sheets for checking 100 urban and 100 natural labels
by eye (saved to Drive/figures)
import matplotlib.pyplot as plt
for sc in ["urban", "natural"]:
    out = f"{DRIVE_DIR}/figures/scene_check_{sc}.png"
    if os.path.exists(out):
        continue
    pool = test[test.scene == sc]
    pick = pool.sample(min(100, len(pool)), random_state=0)
    fig, axs = plt.subplots(10, 10, figsize=(20, 20))
    for k, (ax, i) in enumerate(zip(axs.flat, pick.id)):
        ax.imshow(store.get(i)); ax.set_title(str(k + 1), fontsize=9);
ax.axis("off")
```

```
    fig.suptitle(f"Labelled '{sc}' by the scene classifier (100 random
test images)", fontsize=16)
    fig.tight_layout(); fig.savefig(out, dpi=60); plt.close(fig)
    pick.assign(number=range(1, len(pick) + 1))[["number", "id",
"country", "conf"]].to_csv(out.replace(".png", ".csv"), index=False)
print("Open figures/scene_check_urban.png and scene_check_natural.png in
Drive and count the images that break the rule.")
```

```
Open figures/scene_check_urban.png and scene_check_natural.png in Drive
and count the images that break the rule.
```

In [66]:

```
# Model check: the starting weights are the official Hugging Face files,
unchanged
from huggingface_hub import HfApi
from transformers import AutoModelForZeroShotImageClassification,
AutoModel, AutoModelForImageClassification
official = {
    "clip":
AutoModelForZeroShotImageClassification.from_pretrained("openai/clip-
vit-base-patch16").vision_model,
    "vit": AutoModel.from_pretrained("google/vit-base-patch16-224-
in21k"),
    "resnet":
AutoModelForImageClassification.from_pretrained("microsoft/resnet-
50").resnet,
}
check = {}
for k, ref_model in official.items():
    ref, mine = ref_model.state_dict(), te.Net(k).backbone.state_dict()
    same = sum(torch.equal(v, ref[n]) for n, v in mine.items() if n in
ref)
    check[k] = [same, len(mine)]
    print(f"{te.NAMES[k]}: {same} of {len(mine)} weight tensors
identical to the official Hugging Face file")
api = HfApi()
versions = {ck: api.model_info(ck).sha for ck in ["openai/clip-vit-base-
patch16", "google/vit-base-patch16-224-in21k", "microsoft/resnet-50"]}
versions["osv5m/osv5m (dataset)"] = api.dataset_info("osv5m/osv5m").sha
json.dump({"identical_tensors": check, "versions": versions}, open(f"
{DRIVE_DIR}/results/model_check.json", "w"), indent=1)
print("Official Hugging Face versions used:", versions)
del official
```

```
CLIP ViT-B/16: 199 of 199 weight tensors identical to the official Hugging
Face file
ViT-B/16 (IN-21k): 198 of 198 weight tensors identical to the official
Hugging Face file
ResNet-50: 318 of 318 weight tensors identical to the official Hugging
Face file
Official Hugging Face versions used: {'openai/clip-vit-base-patch16':
'57c216476eefef5ab752ec549e440a49ae4ae5f3', 'google/vit-base-patch16-224-
in21k': 'b4569560a39a0f1af58e3ddaf17facf20ab919b0', 'microsoft/resnet-50':
'34c2154c194f829b11125337b98c8f5f9965ff19', 'osv5m/osv5m (dataset)':
'cff33609b56b54d8743b7ee7a416eb8433e9a681'}
```

D. Learning-rate search (same for every model, seed 0)

In [67]:

```
def make_loaders(kind, seed):
    cfg = te.MODELS[kind]
    tr_ds = te.ImageSet(store, part["train"].id, part["train"].label,
cfg["mean"], cfg["std"], train=True)
    va_ds = te.ImageSet(store, part["val"].id, part["val"].label,
cfg["mean"], cfg["std"])
    train_fn = lambda ep: te.loader(tr_ds, BATCH, True, seed=seed * 1000
+ ep, drop_last=True)
    return train_fn, te.loader(va_ds, 128, False)

def run_json(name):
    p = f"{DRIVE_DIR}/runs/{name}.json"
    return json.load(open(p)) if os.path.exists(p) else None

def stage1(kind, seed):
    # head-only stage; returns the head weights at the end of stage 1
    name = f"{kind}_seed{seed}_stage1"
    hp = f"{DRIVE_DIR}/models/{name}_head.pt"
    if run_json(name) and os.path.exists(hp):
        return torch.load(hp)
    log(f"{name}: training the new head on the frozen backbone")
    te.set_seed(seed)
    model = te.Net(kind).to(device)
    train_fn, val_dl = make_loaders(kind, seed)
    t0 = time.time()
    hist, _ = te.run_stage(model, 1, train_fn, val_dl, STAGE1_EPOCHS,
STAGE1_LR, device,
                           ckpt_path=f"/content/{name}.ckpt", log=log)
    head = {k: v.cpu() for k, v in model.head.state_dict().items()}
    torch.save(head, hp)
    json.dump({"hist": hist, "minutes": (time.time() - t0) / 60},
open(f"{DRIVE_DIR}/runs/{name}.json", "w"))
    if os.path.exists(f"/content/{name}.ckpt"):
        os.remove(f"/content/{name}.ckpt")
    del model; torch.cuda.empty_cache()
    return head

def stage2(kind, seed, lr, epochs, name, keep_best):
    if run_json(name):
        return run_json(name)
    head = stage1(kind, seed)
    log(f"{name}: fine-tuning all weights, lr {lr:g}, {epochs} epochs")
    te.set_seed(seed)
    model = te.Net(kind).to(device)
    model.head.load_state_dict(head)
    train_fn, val_dl = make_loaders(kind, seed)
    t0 = time.time()
    ck = f"{DRIVE_DIR}/runs/{name}.ckpt"
    hist, best_state = te.run_stage(model, 2, train_fn, val_dl, epochs,
lr, device, ckpt_path=ck, log=log)
    if keep_best:
        torch.save(best_state, f"{DRIVE_DIR}/models/{name}_best.pt")
    res = {"kind": kind, "seed": seed, "lr": lr, "hist": hist,
"best_val": max(h["val_acc"] for h in hist),
           "best_epoch": int(np.argmax([h["val_acc"] for h in hist])) +
1, "minutes": (time.time() - t0) / 60,
           "gpu": gpu, "torch": torch.__version__, "transformers":
transformers.__version__}
    json.dump(res, open(f"{DRIVE_DIR}/runs/{name}.json", "w"))
```

```
    if os.path.exists(ck):
        os.remove(ck)
    del model; torch.cuda.empty_cache()
    return res

lr_choice_file = f"{DRIVE_DIR}/runs/lr_choice.json"
LR = json.load(open(lr_choice_file)) if os.path.exists(lr_choice_file)
else {}
for kind in MODELS_TO_RUN:
    if kind in LR:
        continue
    scores = {lr: stage2(kind, 0, lr, SEARCH_EPOCHS, f"
{kind}_search_lr{lr:g}", False)["best_val"] for lr in LR_GRID}
    best = max(scores, key=scores.get)
    if best in LR_EXTEND:
        lr2 = LR_EXTEND[best]
        scores[lr2] = stage2(kind, 0, lr2, SEARCH_EPOCHS, f"
{kind}_search_lr{lr2:g}", False)["best_val"]
        best = max(scores, key=scores.get)
    LR[kind] = best
    log(f"{kind}: validation accuracy by learning rate {({f'{k:g}':
round(v, 4) for k, v in scores.items()})} -> chosen {best:g}")
    json.dump(LR, open(lr_choice_file, "w"))
print("chosen learning rates:", LR)
```

```
chosen learning rates: {'clip': 1e-05, 'vit': 0.0003, 'resnet': 0.001}
```

E. Final runs: two-stage fine-tuning, 3 seeds per model

In [68]:

```
def test_predict(kind, seed):
    # test-set predictions for one finished run: computed once, right
after its training, and only analysed in section G
    out = f"{DRIVE_DIR}/preds/{kind}_seed{seed}.npz"
    if os.path.exists(out) or not run_json(f"{kind}_seed{seed}_final"):
        return
    cfg = te.MODELS[kind]
    dl = te.loader(te.ImageSet(store, test.id, test.label, cfg["mean"],
cfg["std"]), 128, False)
    model = te.Net(kind).to(device)
    model.head.load_state_dict(torch.load(f"
{DRIVE_DIR}/models/{kind}_seed{seed}_stage1_head.pt"))
    head_logits, _ = te.predict(model, dl, device)
# head-only baseline (stage 1)
    model.load_state_dict({k: v.float() for k, v in torch.load(f"
{DRIVE_DIR}/models/{kind}_seed{seed}_final_best.pt").items()})
    logits, y = te.predict(model, dl, device)
# selected stage-2 checkpoint
    np.savez_compressed(out, logits=logits.astype(np.float16),
head_logits=head_logits.astype(np.float16), y=y)
    log(f"{kind} seed {seed}: test predictions saved")
    del model; torch.cuda.empty_cache()

for seed in SEEDS:                      # seed 0 for all three models
first, then seed 1, then seed 2
    for kind in MODELS_TO_RUN:
        r = stage2(kind, seed, LR[kind], STAGE2_EPOCHS, f"
{kind}_seed{seed}_final", True)
        log(f"{kind} seed {seed}: best validation accuracy
```

```
{r['best_val']:.4f} at epoch {r['best_epoch']} ({r['minutes']:.0f}
min)")
        test_predict(kind, seed)
```

```
clip seed 0: best validation accuracy 0.8586 at epoch 8 (43 min)
vit seed 0: best validation accuracy 0.7550 at epoch 8 (20 min)
resnet seed 0: best validation accuracy 0.7260 at epoch 7 (31 min)
clip seed 1: best validation accuracy 0.8588 at epoch 8 (56 min)
vit seed 1: best validation accuracy 0.7500 at epoch 8 (53 min)
resnet seed 1: best validation accuracy 0.7224 at epoch 8 (31 min)
clip seed 2: best validation accuracy 0.8594 at epoch 8 (57 min)
vit seed 2: best validation accuracy 0.7460 at epoch 8 (53 min)
resnet seed 2: best validation accuracy 0.7306 at epoch 7 (32 min)
```

F. Test predictions (each final model is run on the test set once, straight after its training)

In [69]:
```
done = [(k, s) for s in SEEDS for k in MODELS_TO_RUN if run_json(f"
{k}_seed{s}_final")]
print(f"{len(done)} of {len(SEEDS) * len(MODELS_TO_RUN)} final runs
finished")
for kind, seed in done:
    test_predict(kind, seed)
```

```
9 of 9 final runs finished
```

G. Results, statistics and figures

In [70]:
```
C = dp.COUNTRIES
avail = [(k, s) for k, s in done if os.path.exists(f"
{DRIVE_DIR}/preds/{k}_seed{s}.npz")]
seeds_ok = [s for s in SEEDS if all((k, s) in avail for k in
MODELS_TO_RUN)]
P, H = {}, {}
for k, s in avail:
    z = np.load(f"{DRIVE_DIR}/preds/{k}_seed{s}.npz")
    P[(k, s)], H[(k, s)] = z["logits"].argmax(1),
z["head_logits"].argmax(1)
y = test.label.values
R = {"seeds": seeds_ok, "lr": LR, "eligible": ELIGIBLE, "models": {}}
for k in MODELS_TO_RUN:
    ft = [an.basic_metrics(P[(k, s)], y, C) for s in seeds_ok]
    ho = [an.basic_metrics(H[(k, s)], y, C) for s in seeds_ok]
    R["models"][k] = {
        "top1_mean": float(np.mean([m["top1"] for m in ft])), "top1_sd":
float(np.std([m["top1"] for m in ft], ddof=1)) if len(ft) > 1 else 0.0,
        "f1_mean": float(np.mean([m["macro_f1"] for m in ft])), "f1_sd":
float(np.std([m["macro_f1"] for m in ft], ddof=1)) if len(ft) > 1 else
0.0,
        "head_only_top1_mean": float(np.mean([m["top1"] for m in ho])),
        "recall": {c: float(np.mean([m["recall"][c] for m in ft])) for c
in C},
        "top1_per_seed": [m["top1"] for m in ft],
    }
# H1 and RQ1: McNemar per seed
R["mcnemar"] = {f"{a}_vs_{b}": [an.mcnemar(P[(a, s)] == y, P[(b, s)] ==
y) for s in seeds_ok]
                for a, b in [("clip", "vit"), ("clip", "resnet"),
```

```
("vit", "resnet")]}
# H2: urban-natural drop on eligible countries, sequence bootstrap
pooled = len(ELIGIBLE) < 10
test_sc = test.copy()
for k in MODELS_TO_RUN:
    R["models"][k]["drop_pp"] = an.drop_estimate(test_sc, P, k,
seeds_ok, ELIGIBLE, pooled)[0]
st = an.scene_table(test_sc, P, MODELS_TO_RUN, seeds_ok)
st_e = st[st.country.isin(ELIGIBLE)] if not pooled else st
acc_sc = st_e.groupby(["model", "scene"]).acc.mean().unstack()
for k in MODELS_TO_RUN:
    R["models"][k]["urban_acc"], R["models"][k]["natural_acc"] =
float(acc_sc.loc[k, "urban"]), float(acc_sc.loc[k, "natural"])
    R["models"][k]["natural_urban_ratio"] = float(acc_sc.loc[k,
"natural"] / acc_sc.loc[k, "urban"])
bs = an.h2_bootstrap(test_sc, P, seeds_ok, ELIGIBLE, pooled, B=2000)
R["h2"] = {"diff_pp": R["models"]["clip"]["drop_pp"] - R["models"]
["vit"]["drop_pp"], "ci95": [bs["lo"], bs["hi"]], "pooled": pooled}
# check 3: confident scene labels only
conf_mask = test_sc.conf.values >= 0.6
tc = test_sc[conf_mask].reset_index(drop=True)
el_c = an.eligible_countries(tc); pooled_c = len(el_c) < 10
Pc = {key: v[conf_mask] for key, v in P.items()}
dc = {k: an.drop_estimate(tc, Pc, k, seeds_ok, el_c, pooled_c)[0] for k
in ["clip", "vit"]}
bsc = an.h2_bootstrap(tc, Pc, seeds_ok, el_c, pooled_c, B=2000, seed=1)
R["h2_confident"] = {"n_images": int(conf_mask.sum()), "eligible": el_c,
"diff_pp": dc["clip"] - dc["vit"], "ci95": [bsc["lo"], bsc["hi"]]}
# secondary: test images whose contributor is not in the training sample
(pooled)
unseen = ~test.creator_id.isin(set(part["train"].creator_id.dropna())) &
test.creator_id.notna()
R["unseen_contributors"] = {"n_images": int(unseen.sum()),
                            **{k: float(np.mean([(P[(k, s)]
[unseen.values] == y[unseen.values]).mean() for s in seeds_ok])) for k
in MODELS_TO_RUN}}
# RQ3: confusions and look-alike groups (fine-tuned models, all seeds
pooled)
for k in MODELS_TO_RUN:
    pa = np.concatenate([P[(k, s)] for s in seeds_ok]); ya = np.tile(y,
len(seeds_ok))
    conf, cm = an.top_confusions(pa, ya, C, k=8)
    R["models"][k]["top_confusions"] = [[int(n), a, b] for n, a, b in
conf]
    R["models"][k]["errors_within_lookalike_groups"] =
an.lookalike_share(pa, ya, C)
    np.save(f"{DRIVE_DIR}/results/confusion_{k}.npy", cm)
# secondary: per country, share of test images whose contributor is in
the training sample, against recall
from scipy.stats import spearmanr
seen_c =
test.creator_id.isin(set(part["train"].creator_id.dropna())).values
share = pd.Series(seen_c).groupby(test.country.values).mean()
R["contributor_share_by_country"] = {c: float(share[c]) for c in C}
R["contributor_share_vs_recall"] = {k: float(spearmanr([share[c] for c
in C], [R["models"][k]["recall"][c] for c in C]).correlation) for k in
MODELS_TO_RUN}
json.dump(R, open(f"{DRIVE_DIR}/results/results.json", "w"), indent=1)
pd.DataFrame({"id": test.id, "country": test.country, "scene":
test.scene, "scene_conf": test.conf.round(3),
```

```
            **{f"{k}_seed{s}": [C[i] for i in P[(k, s)]] for k, s in
avail}}).to_csv(f"{DRIVE_DIR}/results/test_predictions.csv",
index=False)
st.to_csv(f"{DRIVE_DIR}/results/scene_accuracy_by_country.csv",
index=False)
```

In [71]:
```
# Summary
print(f"seeds used: {seeds_ok} | chosen learning rates: {LR}")
for k in MODELS_TO_RUN:
    m = R["models"][k]
    print(f"{te.NAMES[k]:20s} top-1 {100 * m['top1_mean']:.2f} ± {100 *
m['top1_sd']:.2f} | macro-F1 {100 * m['f1_mean']:.2f} | head-only {100 *
m['head_only_top1_mean']:.2f} "
          f"| urban {100 * m['urban_acc']:.1f} natural {100 *
m['natural_acc']:.1f} drop {m['drop_pp']:.1f} pp | in look-alike groups
{100 * m['errors_within_lookalike_groups']:.0f}% of errors")
for s, r in zip(seeds_ok, R["mcnemar"]["clip_vs_vit"]):
    print(f"H1 seed {s}: CLIP-only correct {r['a_only']}, ViT-only
correct {r['b_only']}, chi2 {r['chi2']:.1f}, p {r['p']:.2g}")
print(f"H2: drop difference CLIP - ViT = {R['h2']['diff_pp']:.2f} pp,
95% CI [{R['h2']['ci95'][0]:.2f}, {R['h2']['ci95'][1]:.2f}] (pooled=
{R['h2']['pooled']})")
print(f"H2 (confident labels only, {R['h2_confident']['n_images']}
images): {R['h2_confident']['diff_pp']:.2f} pp, CI [{R['h2_confident']
['ci95'][0]:.2f}, {R['h2_confident']['ci95'][1]:.2f}]")
print("unseen contributors:", R["unseen_contributors"])
print("per-country share of test images from training contributors vs.
recall (Spearman):", {k: round(x, 2) for k, x in
R["contributor_share_vs_recall"].items()})
```

```
seeds used: [0, 1, 2] | chosen learning rates: {'clip': 1e-05, 'vit':
0.0003, 'resnet': 0.001}
CLIP ViT-B/16        top-1 75.52 ± 0.11 | macro-F1 75.17 | head-only 64.40
| urban 74.9 natural 72.1 drop 2.8 pp | in look-alike groups 17% of errors
ViT-B/16 (IN-21k)    top-1 64.32 ± 0.14 | macro-F1 63.82 | head-only 34.20
| urban 58.9 natural 60.8 drop -1.9 pp | in look-alike groups 13% of
errors
ResNet-50            top-1 62.13 ± 0.14 | macro-F1 61.61 | head-only 30.85
| urban 55.7 natural 58.6 drop -2.9 pp | in look-alike groups 12% of
errors
H1 seed 0: CLIP-only correct 4051, ViT-only correct 1261, chi2 1464.3, p 0
H1 seed 1: CLIP-only correct 4107, ViT-only correct 1244, chi2 1530.8, p 0
H1 seed 2: CLIP-only correct 4097, ViT-only correct 1350, chi2 1384.3, p
5.3e-303
H2: drop difference CLIP - ViT = 4.66 pp, 95% CI [3.04, 6.29]
(pooled=False)
H2 (confident labels only, 14568 images): 4.76 pp, CI [2.73, 6.85]
unseen contributors: {'n_images': 3729, 'clip': 0.5379458299812282, 'vit':
0.37498882631625996, 'resnet': 0.35943505855010277}
per-country share of test images from training contributors vs. recall
(Spearman): {'clip': 0.74, 'vit': 0.83, 'resnet': 0.82}
```

In [72]:
```
# Figures (saved to Drive/figures)
import matplotlib.pyplot as plt
plt.rcParams.update({"font.size": 11, "axes.spines.top": False,
"axes.spines.right": False})
FIG = f"{DRIVE_DIR}/figures"
cols = {"clip": "#2F6FBF", "vit": "#7F7F7F", "resnet": "#C55A11"}
names = [te.NAMES[k] for k in MODELS_TO_RUN]
```

```
# 1 accuracy: head-only vs fine-tuned
fig, ax = plt.subplots(figsize=(7, 4))
x = np.arange(len(MODELS_TO_RUN))
ho = [100 * R["models"][k]["head_only_top1_mean"] for k in
MODELS_TO_RUN]
ft = [100 * R["models"][k]["top1_mean"] for k in MODELS_TO_RUN]
sd = [100 * R["models"][k]["top1_sd"] for k in MODELS_TO_RUN]
ax.bar(x - 0.18, ho, 0.36, color="#A9C4E8", label="Head only (stage 1)")
ax.bar(x + 0.18, ft, 0.36, yerr=sd, color="#2F6FBF", capsize=4,
label="Fine-tuned (stage 2)")
for i in range(len(x)):
    ax.text(x[i] - 0.18, ho[i] + 1.5, f"{ho[i]:.1f}", ha="center",
fontsize=9)
    ax.text(x[i] + 0.18, ft[i] + sd[i] + 1.5, f"{ft[i]:.1f}",
ha="center", fontsize=9)
ax.axhline(4, ls="--", c="k", lw=1); ax.text(2.45, 5, "chance 4%",
ha="right", fontsize=9)
ax.set_ylim(0, 90); ax.set_xticks(x, names); ax.set_ylabel("Top-1
accuracy on the test set (%)"); ax.legend(frameon=False, loc="upper
right")
fig.tight_layout(); fig.savefig(f"{FIG}/fig1_accuracy.png", dpi=200);
plt.close(fig)

# 2 urban vs natural
fig, ax = plt.subplots(figsize=(7, 4))
ua = [100 * R["models"][k]["urban_acc"] for k in MODELS_TO_RUN]
na = [100 * R["models"][k]["natural_acc"] for k in MODELS_TO_RUN]
ax.bar(x - 0.18, ua, 0.36, color="#2F6FBF", label="Urban")
ax.bar(x + 0.18, na, 0.36, color="#4E8A3A", label="Natural")
for i, k in enumerate(MODELS_TO_RUN):
    ax.text(i, max(ua[i], na[i]) + 2, f"urban minus
natural\n{R['models'][k]['drop_pp']:+.1f} points", ha="center",
fontsize=9)
ax.set_ylim(0, 90); ax.set_xticks(x, names); ax.set_ylabel("Top-1
accuracy (%), 21 eligible countries"); ax.legend(frameon=False,
loc="upper right")
fig.tight_layout(); fig.savefig(f"{FIG}/fig2_urban_natural.png",
dpi=200); plt.close(fig)

# 3 per-country recall heatmap
rec = np.array([[100 * R["models"][k]["recall"][c] for c in C] for k in
MODELS_TO_RUN])
order = np.argsort(-rec.mean(0))
fig, ax = plt.subplots(figsize=(11, 2.6))
im = ax.imshow(rec[:, order], cmap="Blues", vmin=0, vmax=100,
aspect="auto")
ax.set_xticks(range(len(C)), [C[i] for i in order]);
ax.set_yticks(range(len(names)), names)
for i in range(rec.shape[0]):
    for j, jj in enumerate(order):
        ax.text(j, i, f"{rec[i, jj]:.0f}", ha="center", va="center",
fontsize=7, color="white" if rec[i, jj] > 60 else "black")
fig.colorbar(im, ax=ax, fraction=0.02, label="Recall (%)")
fig.tight_layout(); fig.savefig(f"{FIG}/fig3_country_recall.png",
dpi=200); plt.close(fig)

# 4 confusion matrix (CLIP, all seeds)
cm = np.load(f"{DRIVE_DIR}/results/confusion_clip.npy"); cmn = 100 * cm
/ cm.sum(1, keepdims=True)
fig, ax = plt.subplots(figsize=(8, 7))
```

```
im = ax.imshow(cmn, cmap="Blues", vmin=0, vmax=100)
ax.set_xticks(range(len(C)), C, rotation=90, fontsize=8);
ax.set_yticks(range(len(C)), C, fontsize=8)
ax.set_xlabel("Predicted country"); ax.set_ylabel("True country");
fig.colorbar(im, ax=ax, fraction=0.04, label="% of true country")
fig.tight_layout(); fig.savefig(f"{FIG}/fig4_confusion_clip.png",
dpi=200); plt.close(fig)

# 5 training curves and 6 learning-rate search
fig, ax = plt.subplots(figsize=(7, 4))
for k in MODELS_TO_RUN:
    for s in seeds_ok:
        h = run_json(f"{k}_seed{s}_final")["hist"]
        ax.plot([e["epoch"] for e in h], [100 * e["val_acc"] for e in
h], c=cols[k], alpha=0.8, label=te.NAMES[k] if s == seeds_ok[0] else
None)
ax.set_xlabel("Stage-2 epoch"); ax.set_ylabel("Validation accuracy
(%)"); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig(f"{FIG}/fig5_training_curves.png",
dpi=200); plt.close(fig)
fig, ax = plt.subplots(figsize=(7, 4))
for k in MODELS_TO_RUN:
    rs = sorted([(r["lr"], r["best_val"]) for r in map(lambda p:
json.load(open(p)), glob.glob(f"
{DRIVE_DIR}/runs/{k}_search_lr*.json"))])
    ax.plot([a for a, _ in rs], [100 * b for _, b in rs], "o-",
c=cols[k], label=te.NAMES[k])
ax.set_xscale("log"); ax.set_xlabel("Stage-2 learning rate");
ax.set_ylabel("Validation accuracy (%)"); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig(f"{FIG}/fig6_lr_search.png", dpi=200);
plt.close(fig)

# 7 sample outputs
rng = np.random.RandomState(3)
pick = list(test[test.scene == "urban"].sample(6,
random_state=rng).index) + list(test[test.scene == "natural"].sample(6,
random_state=rng).index)
s0 = seeds_ok[0]
fig, axs = plt.subplots(3, 4, figsize=(14, 9))
for ax, i in zip(axs.flat, pick):
    ax.imshow(store.get(test.id[i])); ax.axis("off")
    lines = [f"true {C[y[i]]} ({test.scene[i]})"] + [f"{k}: {C[P[(k,
s0)][i]]}" + (" ✓" if P[(k, s0)][i] == y[i] else " ✗") for k in
MODELS_TO_RUN]
    ax.set_title("\n".join(lines), fontsize=9)
fig.suptitle("Sample test predictions (OSV-5M test images, Mapillary
contributors, CC BY-SA)", fontsize=11)
fig.tight_layout(); fig.savefig(f"{FIG}/fig7_samples.png", dpi=150);
plt.close(fig)

# 8 H2 bootstrap distribution
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.hist(bs["samples"], bins=50, color="#2F6FBF", alpha=0.8)
for v in R["h2"]["ci95"]:
    ax.axvline(v, c="k", ls="--", lw=1)
ax.axvline(0, c="#C55A11", lw=1.5)
ax.set_xlabel("Drop difference, CLIP minus ImageNet-21k ViT (percentage
points)"); ax.set_ylabel("Bootstrap samples")
fig.tight_layout(); fig.savefig(f"{FIG}/fig8_h2_bootstrap.png",
```

```
dpi=200); plt.close(fig)
print("figures saved:", sorted(os.listdir(FIG)))
```

```
figures saved: ['fig1_accuracy.png', 'fig2_urban_natural.png',
'fig3_country_recall.png', 'fig4_confusion_clip.png',
'fig5_training_curves.png', 'fig6_lr_search.png', 'fig7_samples.png',
'fig8_h2_bootstrap.png', 'scene_check_natural.csv',
'scene_check_natural.png', 'scene_check_urban.csv',
'scene_check_urban.png']
```